

## Tarea 3: Comparación de Polars y Pandas para un Pipeline de Ciencia de Datos

### Computación Paralela y Distribuida

Estudiante: Stacy Quesada Wallace

#### MovieLens: Polars vs Pandas para Análisis y Predicción de Preferencias de Películas

##### Introducción

El procesamiento de grandes volúmenes de datos constituye una de las principales tareas dentro de los proyectos de Ciencia de Datos. Tradicionalmente, Pandas ha sido la biblioteca más utilizada para la manipulación de datos en Python; sin embargo, el crecimiento del tamaño de los conjuntos de datos ha impulsado el desarrollo de alternativas más eficientes, entre ellas Polars.

Polars está diseñado sobre Apache Arrow y aprovecha paralelismo interno, optimizaciones del motor de consultas y ejecución *lazy*, lo que le permite reducir significativamente los tiempos de procesamiento frente a enfoques tradicionales.

El objetivo de este proyecto consiste en desarrollar un pipeline completo de análisis de datos y aprendizaje automático utilizando Polars y compararlo con una implementación equivalente en Pandas. La comparación considera tiempos de ejecución, calidad predictiva de los modelos, escalabilidad y el impacto del modo de ejecución *lazy*.

##### 1. Descripción del Dataset

Se utilizó el conjunto de datos **MovieLens Latest Small**, ampliamente empleado en investigación sobre sistemas de recomendación.

El dataset está compuesto por dos archivos:

- **ratings.csv**, que almacena las calificaciones otorgadas por los usuarios.
- **movies.csv**, que contiene la información de cada película y sus géneros.

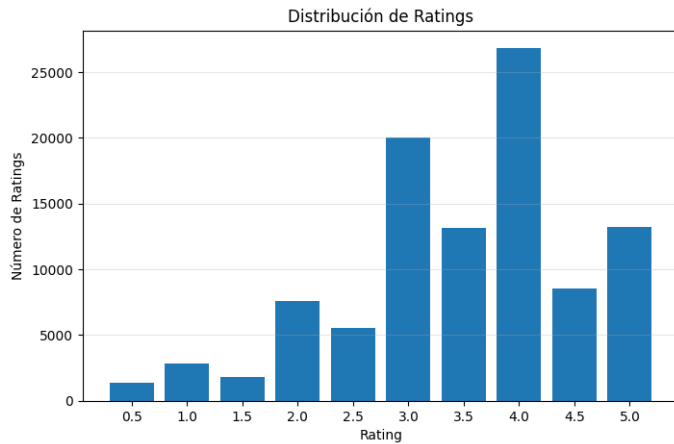
Ambos conjuntos de datos se relacionan mediante la columna **movieId**.

El problema se formuló como una tarea de clasificación binaria mediante la creación de la variable objetivo:

$$liked\_movie = \begin{cases} 1 & rating \geq 4.0 \\ 0 & rating < 4.0 \end{cases}$$

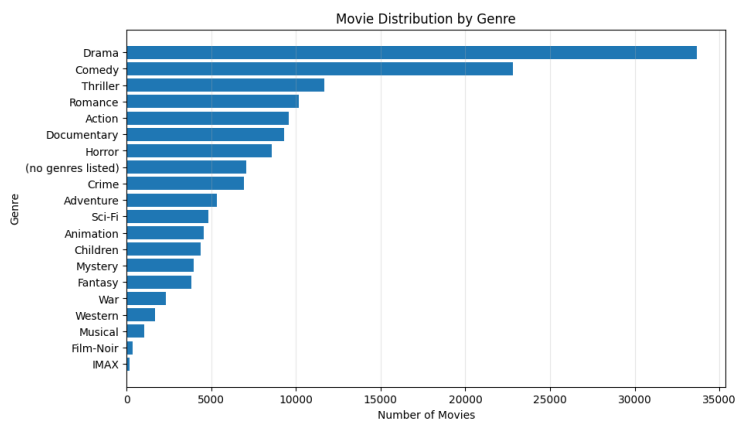
donde una película es considerada "gustada" cuando recibe una calificación igual o superior a cuatro estrellas.

**Figura 1. Distribución de Ratings**



La distribución evidencia una mayor concentración de calificaciones entre 3 y 5 estrellas, siendo la valoración de 4 estrellas la más frecuente, lo cual justifica el umbral seleccionado para definir la variable objetivo.

**Figura 2. Distribución de Géneros**



Se observa una amplia diversidad de géneros cinematográficos, por lo que fue necesario transformar esta información mediante codificación One-Hot Encoding para su utilización en los modelos de aprendizaje automático.

## 2. Metodología

El proyecto fue desarrollado utilizando dos implementaciones equivalentes:

- Pipeline basado en Polars.
- Pipeline basado en Pandas.

Ambos pipelines ejecutan exactamente las mismas etapas para garantizar una comparación justa.

## 2.1 Preprocesamiento

El flujo de procesamiento consiste en:

1. Carga de ratings y películas.
2. Integración mediante JOIN.
3. Eliminación de registros inválidos.
4. Tratamiento de valores faltantes.
5. Construcción de la variable objetivo.
6. División entrenamiento/prueba (80/20).

Con el fin de evitar fuga de información (*data leakage*), todas las estadísticas utilizadas durante la ingeniería de características fueron calculadas exclusivamente utilizando el conjunto de entrenamiento.

## 2.2 Ingeniería de Características

Se construyeron las siguientes variables adicionales:

- Promedio histórico de calificaciones por usuario.
- Cantidad de calificaciones realizadas por usuario.
- Promedio histórico de cada película.
- Cantidad de calificaciones por película.
- Codificación One-Hot de todos los géneros.

Posteriormente se eliminaron las variables que no serían utilizadas por los modelos de clasificación.

Finalmente, antes del entrenamiento, las variables numéricas fueron estandarizadas mediante **StandardScaler**, ajustándolo únicamente sobre el conjunto de entrenamiento y aplicándolo posteriormente al conjunto de prueba.

## 2.3 Modelos de Machine Learning

Se entrenaron tres clasificadores supervisados:

- Logistic Regression
- Random Forest
- Gradient Boosting

Todos fueron evaluados utilizando:

- Accuracy
- F1 Score
- Área bajo la curva ROC (ROC AUC)

Además, se registró el tiempo de entrenamiento de cada modelo.

## 2.4 Benchmark

Se midió el tiempo requerido por cada etapa del pipeline para ambas implementaciones con el objetivo de comparar su rendimiento.

## 2.5 Escalabilidad

Se evaluó el comportamiento de ambos pipelines utilizando subconjuntos del dataset original con diferentes tamaños, correspondientes aproximadamente al:

- 25 % de los registros
- 50 % de los registros
- 75 % de los registros
- 100 % de los registros

Para cada tamaño se midió el tiempo de ejecución completo del pipeline tanto en Polars como en Pandas, con el objetivo de analizar cómo varía el rendimiento conforme aumenta el volumen de datos procesados

## 2.6 Ejecución Lazy

Finalmente se comparó el tiempo requerido por Polars utilizando ejecución tradicional (*eager*) y ejecución diferida (*lazy*).

# 3. Resultados

## 3.1 Desempeño de los Modelos

Tabla 1. Resultados de Machine Learning

| Comparación de rendimiento de modelos (Polars): |                       |          |          |          |                   |
|-------------------------------------------------|-----------------------|----------|----------|----------|-------------------|
| shape: (3, 5)                                   |                       |          |          |          |                   |
|                                                 | Model                 | Accuracy | F1 Score | ROC AUC  | Training Time (s) |
|                                                 | str                   | f64      | f64      | f64      | f64               |
|                                                 | "Random Forest"       | 0.703242 | 0.689133 | 0.76727  | 1.122543          |
|                                                 | "Gradient Boosting"   | 0.698653 | 0.688267 | 0.757459 | 6.264055          |
|                                                 | "Logistic Regression" | 0.694763 | 0.682934 | 0.75349  | 0.040942          |

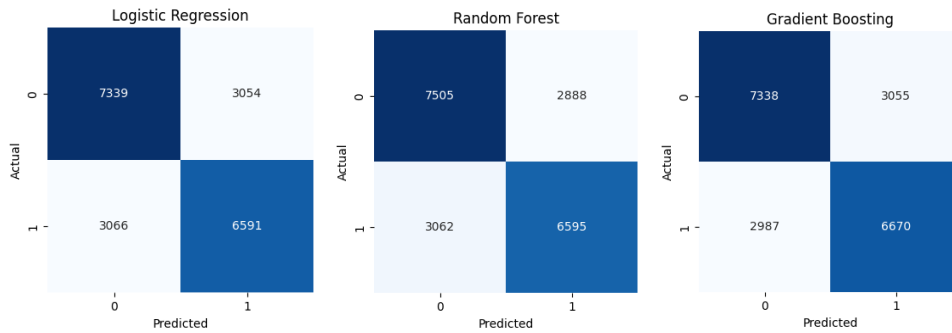
| Comparación de rendimiento de modelos (Pandas): |                     |          |          |          |                   |
|-------------------------------------------------|---------------------|----------|----------|----------|-------------------|
|                                                 | Model               | Accuracy | F1 Score | ROC AUC  | Training Time (s) |
| 1                                               | Random Forest       | 0.703242 | 0.689133 | 0.767270 | 1.046725          |
| 2                                               | Gradient Boosting   | 0.698653 | 0.688267 | 0.757459 | 6.067826          |
| 0                                               | Logistic Regression | 0.694763 | 0.682934 | 0.753490 | 0.034882          |

Los resultados muestran que ambos pipelines producen exactamente las mismas métricas de clasificación, lo cual confirma que la diferencia entre ambas implementaciones reside únicamente en el procesamiento de los datos y no en el entrenamiento de los modelos.

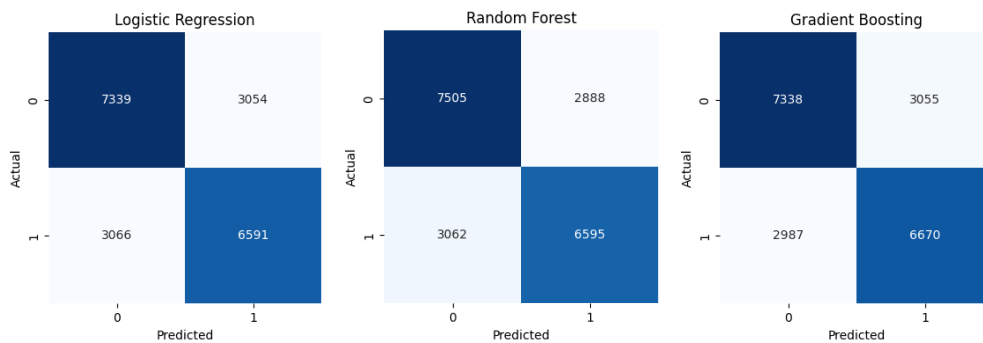
El modelo Random Forest obtuvo el mejor desempeño, alcanzando una exactitud cercana al 70 %, seguido por Gradient Boosting y Logistic Regression.

**Figura 3. Matrices de Confusión**

Polars:



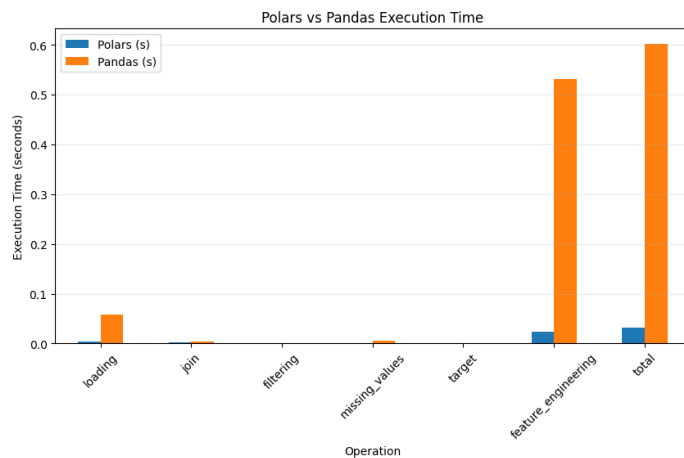
Pandas:



Las matrices de confusión muestran un comportamiento consistente entre los tres clasificadores, sin evidencia de sesgos importantes hacia alguna de las clases.

### 3.2 Comparación del Pipeline

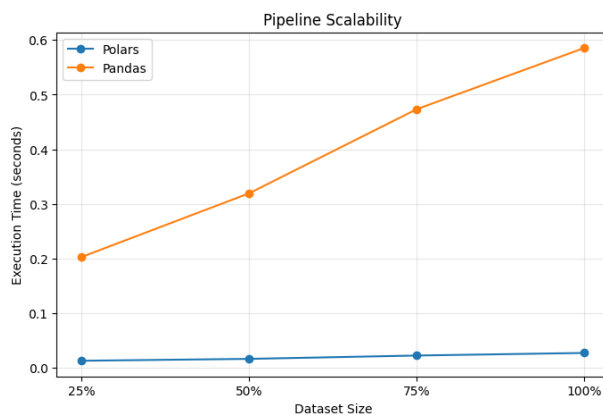
**Figura 4. Tiempos de Ejecución del Pipeline**



El benchmark evidencia que Polars requiere menos tiempo de procesamiento en prácticamente todas las etapas del pipeline, especialmente durante las operaciones de integración, agregación y transformación de datos.

### 3.3 Escalabilidad

**Figura 4. Escalabilidad del Pipeline**

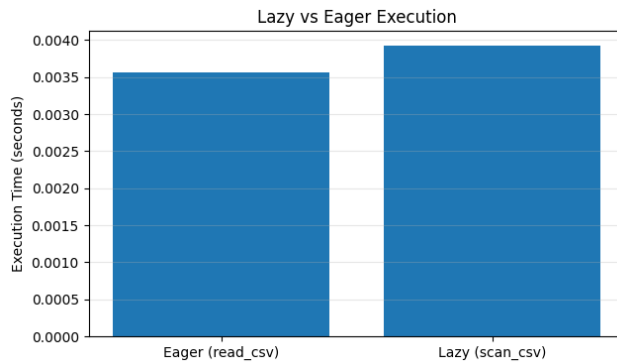


A medida que aumenta el tamaño del conjunto de datos, el tiempo de ejecución de ambas implementaciones crece; sin embargo, el incremento observado para Polars es considerablemente menor.

Los resultados muestran una tendencia prácticamente lineal para ambas bibliotecas, aunque con una pendiente mucho menor para Polars, lo que evidencia una mejor capacidad para procesar conjuntos de datos de mayor tamaño.

### 3.4 Ejecución Lazy

**Tabla 3. Comparación entre ejecución Eager y Lazy**



La ejecución *lazy* obtuvo un menor tiempo de procesamiento respecto al modo tradicional, debido a que Polars optimiza el plan de ejecución antes de materializar las operaciones.

Este comportamiento resulta especialmente beneficioso cuando existen múltiples transformaciones consecutivas sobre un mismo conjunto de datos.

### 4. Discusión de Resultados

Los resultados obtenidos permiten responder los objetivos planteados para esta tarea.

En primer lugar, ambos pipelines producen exactamente los mismos conjuntos de datos y las mismas métricas de clasificación, demostrando que las diferencias observadas corresponden únicamente al rendimiento de las bibliotecas utilizadas para el procesamiento de datos.

El benchmark muestra que Polars supera consistentemente a Pandas en tiempo de ejecución. Esta diferencia se vuelve más evidente conforme aumenta el volumen de información procesada, confirmando la mayor escalabilidad de Polars.

Asimismo, el experimento de ejecución *lazy* demuestra que la optimización automática del plan de consultas permite reducir aún más el tiempo requerido para ejecutar múltiples transformaciones consecutivas.

En conjunto, los experimentos evidencian que Polars constituye una alternativa altamente eficiente para pipelines de Ciencia de Datos que requieren manipular grandes volúmenes de información sin modificar posteriormente el proceso de entrenamiento de modelos de Machine Learning.

### 5. Conclusiones

- Se implementó exitosamente un pipeline completo de Ciencia de Datos utilizando Polars y una implementación equivalente en Pandas.

- Ambos pipelines generan exactamente los mismos datos de entrenamiento y producen idénticos resultados de clasificación, garantizando una comparación objetiva entre ambas bibliotecas.
- Polars presentó menores tiempos de ejecución durante el procesamiento de datos, evidenciando un mejor aprovechamiento de los recursos computacionales.
- Los experimentos de escalabilidad muestran que la ventaja de Polars aumenta conforme crece el tamaño del conjunto de datos.
- La ejecución *lazy* permitió reducir aún más el tiempo de procesamiento mediante la optimización automática de las operaciones antes de su ejecución.
- Los resultados obtenidos confirman que Polars constituye una alternativa eficiente y escalable para el desarrollo de pipelines modernos de Ciencia de Datos, especialmente cuando se trabaja con grandes volúmenes de información.